

Agent SDK / agentic implementation steps

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 11 of 12 · English notes

After RAG, the session defined **agents** and **agentic AI**, then installed the **Claude Agent SDK** (renamed from what the trainer still called Claude Code SDK). GenAI is “prompt in, text (or image) out.” An agent is a packaged worker. Agentic AI is **goal-shaped**: plan, act with tools, reflect, loop until the goal is done or blocked.

Agent versus agentic application

An **agent** in this course is a configurable bundle: some combination of an LLM, tools (in-process or MCP), memory, prompting technique, and optionally RAG. You pick which features that worker needs. A retrieval agent might only fetch and format chunks. The LLM is optional in the trainer’s taxonomy — a worker could be a deterministic tool wrapper — but the interesting agents reason.

Agentic AI orchestrates many such tasks toward a goal. Recruitment example: “Hire a senior Java full-stack engineer, issue an offer, and onboard them.” Subtasks include writing a JD, posting to Naukri and/or an internal portal, screening CVs, scheduling interviews, messaging candidates. Each step can be an agent; the orchestrator decides order, retries, and which integration to use when the goal says “internal only” versus “both boards.”

Plan, act, reflect — not a rules engine

Power Automate-style flows fire on events you pre-wired. Agentic systems take a natural-language goal, produce a plan (reasoning models help), execute actions on integrations you registered up front, capture success / failure / partial results, store them in memory, and **revisit the plan**. Completion is scored against the *goal*, not a single tool call.

- **Short-term memory** — daily blast of emails/WhatsApp, then forget.
- **Long-term memory** — a hire that may take days to a year; configurable retention, not a magic number.
- **Human hand-off** — agent OS should allow agent-to-human transfer when the goal requires it.

“Agent OS” in this classroom

The orchestration layer was named **agent OS**: which agents exist, which system integrations (Naukri, WhatsApp, SMTP) are allowed, how memory works, privacy and guardrails on the incoming goal. In the Claude stack that OS is the **Claude Agent SDK** (Python or TypeScript). `pip install` the current package name from docs — recordings say “Cloud Agent SDK.”

SAME API KEY

A student asked whether the Agent SDK used a different key. Vijay: **no**, the Anthropic API key is the same. Confirm workspaces and rate limits on the live API pages.

Four components in the loop

Piece	Role in Vijay's diagram	Does it touch the disk?
Your application	Calls query with a goal and options; prints the result	Only if you write extra code
Claude Agent SDK	"Nervous system": API calls, tool loop, session state, translating "create a file" into execution	Coordinates; does not replace Claude Code
Claude (LLM)	Thinks: context, plan, which tool, whether the goal is done. Never acts on the machine	No
Claude Code (doer)	Subprocess that reads/writes files, runs bash, executes requested tools, returns stdout to the SDK	Yes

Mnemonic he wanted: **Claude decides, Claude Code does, the SDK loops, your code configures**. Students who thought "three models at once" were corrected: the SDK is a library; the thinker is the Claude model you selected.

Implementation steps in the notebook

1. Install Claude Agent SDK; load `ANTHROPIC_API_KEY`.
2. Choose a model id (check the live models page; classroom ids rot).
3. Build **Claude Agent Options**: allowed tools, model, later memory and permission mode.
4. Call `query(prompt, options)` as an async/sync iterator of messages (system, assistant, tool, result).
5. Enable built-in tools you actually need (Read, Write, Edit, Bash, Glob, Grep, WebSearch, WebFetch, AskUserQuestion were listed — module 12 drills them).
6. Optionally attach hooks (pre/post tool, prompt) and permission policies you already saw in Claude Code sessions.

The first demo query was "what files are in the directory?" without a path: the agent used the working directory, listed a Jupyter notebook and a built-in-tools folder, and offered to dig further. One query hid the multi-step tool loop you wrote by hand in module 10.

Cowork as intuition

Cowork was recalled as an agentic product: attach a folder, ask for file names, ignore nested directories if that is the instruction. The SDK is how you build that shape in Python instead of only in the Cowork GUI.

RAG inside agents

Nothing stops an agent option list from including a custom retrieval tool or an MCP server in front of Atlas. Agentic RAG is “the orchestrator decides when the goal needs documents,” not a new acronym. Guardrails (module 12) wrap the same query so a recruitment goal cannot dump PII into logs.

Official documentation

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

The doer process the Agent SDK drives.

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

The loop the SDK automates.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Third-party tools on the same agent.

- **Managed Agents quickstart**

<https://platform.claude.com/docs/en/managed-agents/quickstart>

Platform-hosted agents; confirm against current product names.

- **Models**

<https://docs.claude.com/en/docs/about-claude/models>

Which id you put in Agent Options.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.